

Rust

ESTR4104

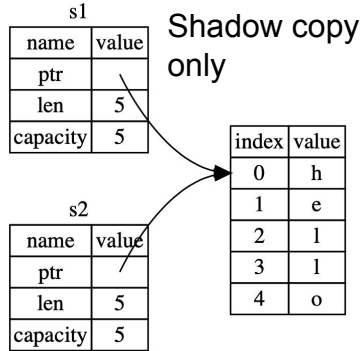
Background

- By design, Rust wants to do auto 'free' for you (as we always forget 'free' the malloc heap memory)
- So, it auto 'drops' your malloc memory for you when your variable is out of scope

Shadow copy

- Shadow copy: efficient
- Shadow copy + auto drop is **unsafe** because when leaving the scope, s2 and s1 will **free the memory twice** (invoking the ***drop trait***; will discuss)
 - dangerous if someone else has reused that piece of memory right after, say, s1, free it
 - Then s2 will also free it but someone else needed it

```
let s1 = String::from("hello");
```



Let use Move semantic, not shadow copy

- Hence **move** = shadow copy + invalidate old pointer

s1	
name	value
ptr	/
len	5
capacity	5

s2	
name	value
ptr	/
len	5
capacity	5

Heap

index	value
0	h
1	e
2	l
3	l
4	o

Move=shadow
copy +
invalidate

```
let s1 = String::from("hello");  
let s2 = s1; //s1 is invalidated
```

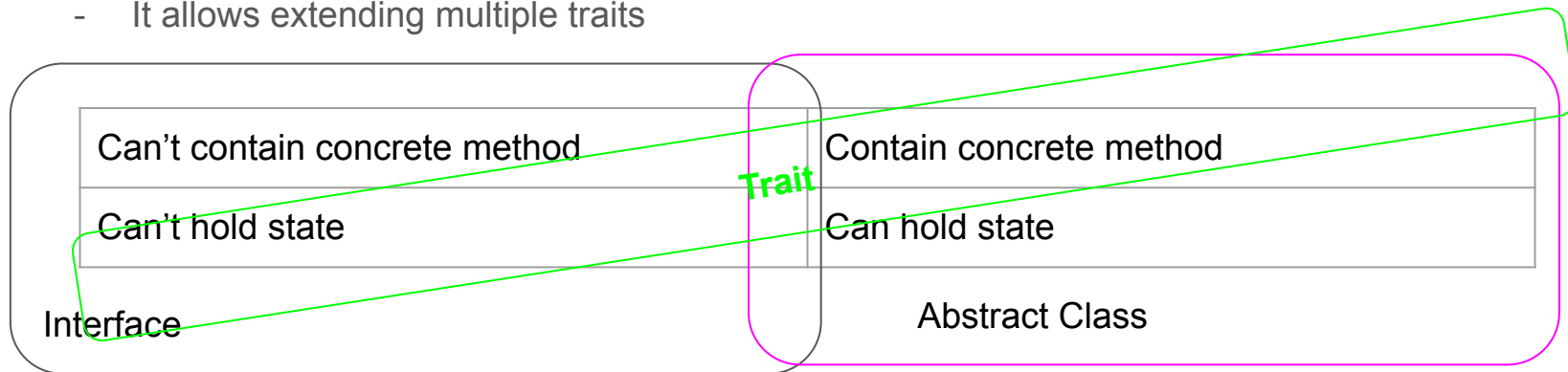
```
println!("{}", world!", s1);  
//Hence, compilation error!
```

Trait

An OO concept

- Like Java Abstract Class
 - Can contain both abstract and concrete methods (with content)
- Unlike Java Abstract Class and Like Interface
 - It itself can't hold any state (variables)
 - It allows extending multiple traits

```
abstract class Shape {  
    int color; //can't in Trait  
  
    // An abstract function  
    abstract void draw();  
}
```



Trait

You implement a trait for a struct and enum

```
pub trait Summary {  
    fn summarize(&self) -> String {  
        String::from("(Read more...)")  
    }  
}  
  
//concrete method give default behavior
```

```
fn main() {  
    let tweet = Tweet {  
        Username: String::from("horse"),  
        content: String::from("yes"),  
        reply: false,  
        retweet: false,  
    };  
  
    println!("1 new tweet: {}",  
tweet.summarize());  
}
```

```
pub struct NewsArticle {  
    pub headline: String,  
    pub location: String,  
    pub author: String,  
    pub content: String,  
}
```

```
impl Summary for NewsArticle { //implement the trait  
    fn summarize(&self) -> String {  
        format!("{}", by {} ({}), self.headline, self.author,  
self.location)  
    }  
}
```

```
pub struct Tweet { //Tweet → Node  
    pub username: String,  
    pub content: String,  
    pub reply: bool,  
    pub retweet: bool,  
}
```

```
impl Summary for Tweet {  
    fn summarize(&self) -> String {  
        format!("{}", self.username, self.content)  
    }  
}
```

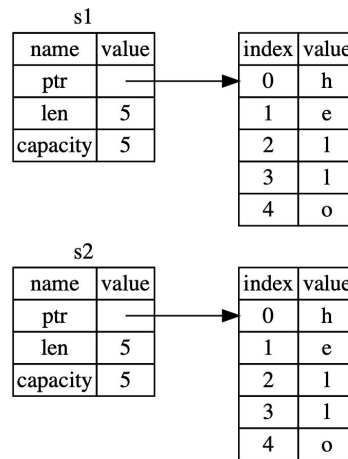
Copy

Shadow copy vs Deep copy

- Shadow copy: copy the first level, no recursive copying inside nested structure
- Deep copy: copy recursively

.clone() is deep copy for objects

```
let s1 = String::from("hello");  
let s2 = s1.clone();
```



Primitives

Primitives implement the Copy trait, so not move

For primitives, shadow/deep are the same

```
let s1 = String::from("hello");  
let s2 = s1; //s1 is invalidated  
  
println!("{}", world!", s1);  
//Hence, compilation error!
```

VS

```
let x: i32 = 42; //on stack  
let y = x; //primitive; is copy;  
not move; x is not invalidated  
  
println!("x: {}", x); // OK  
println!("y: {}", y); // OK
```

Functions

Passing function parameters would then be the same:

If is primitive

- Makes a copy
- Hence caller owns a copy
- The callee owns another copy

Else

- Makes a move

```
fn main() {  
    let s = String::from("hello"); // s comes into scope  
  
    takes_ownership(s);             // s's value moves into the function...  
                                    // ... and so s is no longer valid here  
  
    let x = 5;                      // x comes into scope  
  
    makes_copy(x);                 // x would move into the function,  
                                    // but i32 is Copy, so it's okay to still  
                                    // use x afterward  
  
} // Here, x goes out of scope, then s. But because s's value was moved, nothing  
  // special happens.  
  
fn takes_ownership(some_string: String) { // some_string comes into scope  
    println!("{}", some_string);  
} // Here, some_string goes out of scope and `drop` is called. The backing  
  // memory is freed.  
  
fn makes_copy(some_integer: i32) { // some_integer comes into scope  
    println!("{}", some_integer);  
} // Here, some_integer goes out of scope. some_integer is dropped.
```

Functions and Reference

Here, s1 is a heap string, so
s1 is moved = shadow copy
+ invalidated s1

Hence, s1 can't be used
again in println

To do so, calculate_length
function has to move back s
to s2

```
fn main() {  
    let s1 = String::from("hello");  
  
    let (s2, len) = calculate_length(s1); //s1 is moved!  
  
    println!("The length of '{}' is {}.", s2, len);  
}  
  
fn calculate_length(s: String) -> (String, usize) {  
    let length = s.len(); // len() returns the length of a String  
  
    (s, length)          //← in Rust, return value is the line w/o ";"  
}
```

It is so
inconvenient and
troublesome!

For return value, same rule:

If primitive, use *copy*

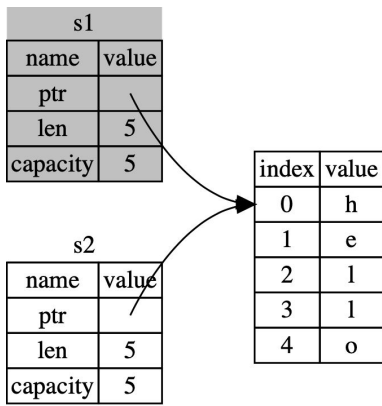
Else use *move* by default (but use copy if the value has
a copy trait)

Ownership

A piece of memory in **heap** has to be owned by only **one** piece of program variable in **stack**: s1 vs s2

That is the owner drops the piece of heap memory when it out of scope

The compiler will flag error to stop you use-after-free



Remember in OS: stack only accepts fix-size that can be known at compile-time

Also, why stack is 'faster' than heap?

Ans: Since heap needs to dynamically find free space in the memory; vs pushing to the stack is easy --- book-keeping of where is the top is easy

Reference (Borrow)

To support cleaner function return, we want:

- 1) shadow copy
- 2) not invalidate the old one
- 3) auto-free on exit scope
- 4) but still ENSURE no double-free problem

Solution: **BORROW and Reference**

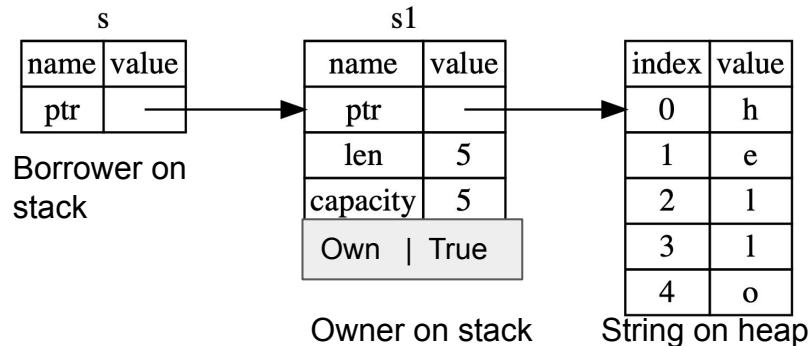
- The reference (borrower) do not take ownership
- So, when `calculate_length` returns (pop from stack), `s` is gone, but won't auto free the heap

```
fn main() {  
    let s1 = String::from("hello");
```

```
    let len = calculate_length(&s1);
```

```
    println!("The length of '{}' is {}.", s1, len);  
}
```

```
fn calculate_length(s: &String) -> usize {  
    s.len()  
}
```



Mutability

- 1) First of all, the string `s1` in the heap != it is mutable.
Hence, we need to make:

let **mut** `s1` = ...

- 2) We need to use **&mut** `s` instead of `&s`:

```
calculate_length(&mut s1); ...
```

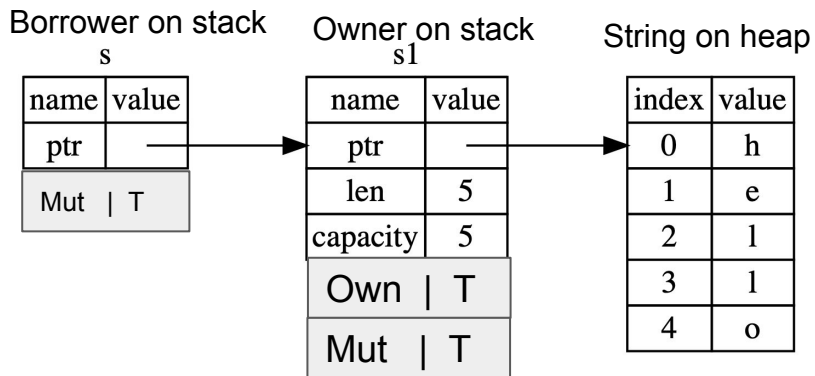
```
fn calculate_length(s: &mut String) -> usize {
```

- 3) The Rust syntax allows at most 1 mutable reference to the same heap object **within the same scope**; and when there is 1 mutable reference, not even immutable reference is allowed

Exclusive write “lock” at programming level!

```
fn main() {  
    let s1 = String::from("hello");  
  
    let len = calculate_length(&s1);  
  
    println!("The length of '{}' is {}.", s1, len);  
}
```

```
fn calculate_length(s: &String) -> usize {  
    s.len()  
}
```



```
fn main() {  
    let mut s1 = String::from("hello");  
    let len = calculate_length(&mut s1);  
  
    println!("The length of '{}' is {}.", s1, len);  
}  
  
fn calculate_length(s: &mut String) -> usize {  
    s.len()  
}
```

```
fn main() { //OK  
    let mut s1 = String::from("hello");  
    let len = calculate_length(&s1); //passing in a  
    non-mutable reference of s1;  
  
    println!("The length of '{}' is {}.", s1, len);  
}
```

```
fn calculate_length(s: &String) -> usize {  
    s.len() //since this reference is non-mutable, you  
    can't call any of its mutable function  
}
```

```
fn main() {  
    let mut s1 = String::from("hello");  
    let len = calculate_length(&s1); //immutable  
    reference  
  
    println!("The length of '{}' is {}.", s1, len);  
}
```

```
fn calculate_length(s: &mut String) -> usize {  
    s.len()  
} //not compilable
```

Hence, it is common to add { } to scope the program in more levels so as to allow you use mut& more while remains safe

```
let mut s = String::from("hello");
```

```
{
```

```
    let r1 = &mut s;
```

```
} // r1 goes out of scope here, so we can make a new reference with no problems.
```

```
let r2 = &mut s;
```

Minimize the size of the critical section!

Furthermore, the compiler will auto-scope for you such that you don't need to add so many `{}` scopes

```
let mut s = String::from("hello");
```

```
let r1 = &s; // immut ref; no problem
```

```
let r2 = &s; // immut ref; no problem
```

```
println!("{}", r1, r2);
```

```
// since variables r1 and r2 will not be used after this point
```

```
// the compiler regards r1 and r2 scope ends here
```

```
let r3 = &mut s; // hence, even &mut is exclusive-lock (x-lock), it is still ok
```

```
println!("{}", r3); //ok!
```

You won't get runtime dangling pointer in Rust

Because the compiler will not let you compile:

Compile error:

- In function `dangle()`,
 - `s` is the owner of the string and will drop the "hello" string from the heap on return
- In `main()`,
 - So, the `reference_to_nothing` is a dangling pointer

```
fn main() {  
    let reference_to_nothing = dangle();  
    println!("{}", reference_to_nothing);  
    //dangling pointer  
    //if permit, will run-time error!  
}  
  
fn dangle() -> &String {  
    let s = String::from("hello");  
  
    &s  
}
```

```
Compile error: can't return  
reference to a local variable s
```

To solve it, return with the ownership using move

“hello” is moved from s to reference_to_nothing

```
fn main() {  
    let reference_to_nothing = dangle();  
    println!("{}", reference_to_nothing);  
    //ok  
}
```

```
fn dangle() -> &String {  
    let s = String::from("hello");
```

```
    &s  
}
```

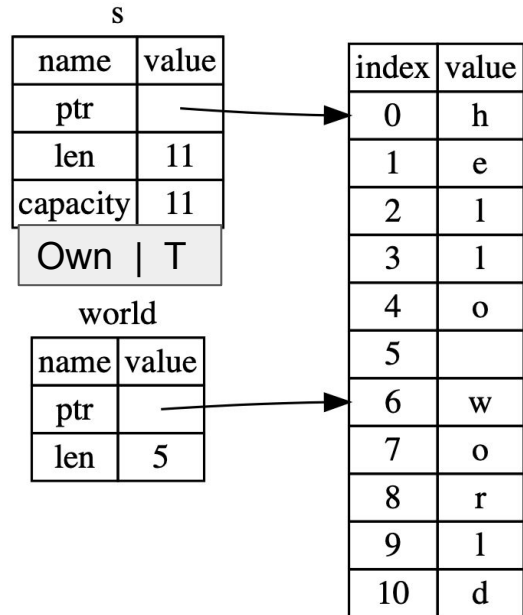
Except Reference, Slice also don't own ownership

- A reference to a part of string
- No ownership

```
let s = String::from("hello world");
```

```
let hello = &s[0..5];
```

```
let world = &s[6..11];
```



Check your knowledge, why s.clear() has error?

```
fn main() {  
    let mut s = String::from("hello world");  
    let word = first_word(&s);  
    s.clear(); // error!  
    println!("the first word is: {}", word);  
}  
  
fn first_word(s: &String) -> &str {  
    let bytes = s.as_bytes();  
  
    for (i, &item) in bytes.iter().enumerate() {  
        if item == b' ' { //b means byte  
            return &s[0..i];  
        }  
    }  
  
    &s[..]  
}
```

Check your knowledge, why s.clear() has error?

Because

- s is a mutable object
- word got returned from the first_word function is an immutable slice to the string
 - (a new reference created and returned by first_word function)
 - word is still active because println in line 4 needs it
- So, when `s.clear();` // it is an error as `s.clear()` first obtains a mutable reference of s and try to clear() it while word is an immutable reference to the same heap **in the same scope**//wr conflict → error

```
fn main() {  
    let mut s = String::from("hello world");  
    let word = first_word(&s);  
    s.clear(); // error!  
    println!("the first word is: {}", word);  
}
```

```
fn first_word(s: &String) -> &str {  
    let bytes = s.as_bytes();  
  
    for (i, &item) in bytes.iter().enumerate() {  
        if item == b' ' { //b means byte  
            return &s[0..i];  
        }  
    }  
  
    &s[..]  
}
```

Box

Similar to malloc

```
fn main() {  
    let x = 5;  
    let y = &x;  
}
```

Memory View

Address 0x90: Integer 5
Address 0x94: Address 0x90

```
fn main() {  
    let x = 5;  
    let y = Box::new(x);  
}
```

Memory View

Address 0x90: Integer 5
Address 0x94: Address 0x9999
... Heap below ...
Address 0x9999: 5

Box is a Smart Pointer

A data structure that

- Includes a raw pointer to the data (can be in stack/heap)
- Owns the data it points to
- Implements Deref trait (can customize its behavior when *ptr)
- Implements Drop trait (can customize its behavior when get dropped)

Safe and Unsafe

rust

```
unsafe fn unsafe_example(ptr: *mut i32) {  
    *ptr = 42; // No compiler checks—your responsibility  
}
```

- Safe code is guaranteed by the compiler to be memory-safe and thread-safe
- Unsafe code opts out of those guarantees, allowing low-level operations at your own risk
- It is not plain to get a raw pointer in Rust, and I don't teach you that here either :-P

Examples

rust

```
unsafe {  
    let ptr: *mut i32 = /* ... */;  
    *ptr = 42; // 1. Dereference raw pointer  
  
    std::slice::from_raw_parts(ptr, 10); // 2. Call unsafe fn  
  
    static mut COUNT: i32 = 0; // 3. Mutable static  
    COUNT += 1;  
  
    // 5. Union access  
    union U { i: i32, f: f32 }  
    let u = U { i: 1 };  
    let f = unsafe { u.f }; // Unsafe union read  
}
```

Core Unsafe Operations

From official docs, these require `unsafe`: [doc.rust-lang+2](#)

1. Dereference raw pointers (`*raw_ptr` or `ptr.read()`).
2. Call unsafe functions/methods (e.g., FFI, `std::slice::from_raw_parts`).
3. Access/modify mutable statics (`static mut`).
4. Implement unsafe traits (e.g., custom ones requiring invariants).
5. Access union fields (non-`#[repr(C)]` unions).

Additional Derived Ones

- Raw pointer arithmetic/methods (e.g., `ptr.add(1)`, `ptr.cast()`).

This explains why PL can be discussed with OS together



ASPLOS

International Conference on Architectural
Support for Programming Languages and
Operating Systems
(ASPLOS)

Cell

Think of it as ‘public setters/getters’ for a private field in Java

More on programming-level and compiler safety than OS level

- A `Cell` is a wrapper around a value that allows you to access and modify the value, but it doesn't own the underlying memory. The memory is still owned by the variable that holds the `Cell`.
- With a `Cell`, you can get a mutable reference to the value inside it, even if the `Cell` itself is immutable. This is possible because `Cell` uses runtime checks to enforce the borrowing rules.

```
struct User {  
    active: bool,  
    username: String,  
    email: String,  
    sign_in_count: Cell<u64>,  
}
```

```
let mut user1 = User {  
    email: String::from("someone@example.com"),  
    username: String::from("someusername123"),  
    active: true,  
    sign_in_count: 1,  
};
```

```
user1.sign_in_count.set(5)
```

Struct and interior mutability and cell

Is it like public method of a class for mutating an object attribute?

```
struct User {  
    active: bool,  
    username: String,  
    email: String,  
    sign_in_count: u64,  
}
```

```
fn main() {  
    let mut user1 = User {  
        email: String::from("someone@example.com"),  
        username: String::from("someusername123"),  
        active: true,  
        sign_in_count: 1,  
    };  
  
    user1.email = String::from("anotheremail@example.com");  
}
```

Note: once you get a mut reference, the whole user1 has every field mutable// rust has no individual private/pub concept

To mutate just one particular field of an **immutable** reference, you can use 'interior mutability'

```
struct User {  
    active: bool,  
    username: String,  
    email: String,  
    sign_in_count: Cell<u64>,  
}
```

```
let mut user1 = User {  
    email: String::from("someone@example.com"),  
    username: String::from("someusername123"),  
    active: true,  
    sign_in_count: 1,  
};  
  
user1.sign_in_count.set(5)
```

```
// Using Box
let mut boxed_value = Box::new(42);
*boxed_value = 50; // Mutate the value inside the Box
```

```
// Using Cell
let cell_value = std::cell::Cell::new(42);
cell_value.set(50); // Mutate the value inside the Cell
```

//the cell location is = simple variable, if inside a function plainly, it's on stack; if the cell is inside a box; it's on heap

Why not just make the whole struct mutable?

Remember, you make it mutable, others can't even get an &mut on it...

Tuple Struct

struct that don't give names to individual attributes

Tuple	Struct	Tuple Struct
<pre>let tup = (500, 6.4, 1);</pre>	<pre>struct User { active: bool, username: String, email: String, sign_in_count: u64, }</pre>	<pre>struct Color(i32, i32, i32); //Tuple Struct struct Point(i32, i32, i32); fn main() { let black = Color(0, 0, 0); let origin = Point(0, 0, 0); }</pre>
One off Won't reuse this structure	With a structure name With attribute names	With a structure name No attribute names

More Smart Pointer - Reference Counting

Let a (nested) structure to have multiple owners and deallocate when last owners' exit

```
use std::rc::Rc;
```

```
fn main() {
```

```
    // Allocate an i32 on the heap, managed by Rc.
```

```
    let a = Rc::new(5);
```

```
    println!("count after creating a = {}", Rc::strong_count(&a)); // 1
```

```
    // Clone `a`: no new i32 is allocated, only the count increases.
```

```
    let b = Rc::clone(&a);
```

```
    println!("count after creating b = {}", Rc::strong_count(&a)); // 2
```

```
{
```

```
    let c = Rc::clone(&a);
```

```
    println!("count after creating c = {}", Rc::strong_count(&a)); // 3
```

```
    // use a, b, c here; they all point to the same 5.
```

```
    println!("a = {}, b = {}, c = {}", a, b, c);
```

```
} // c goes out of scope here, count goes back to 2.
```

```
    println!("count after c is dropped = {}", Rc::strong_count(&a)); // 2
```

```
} // a and then b are dropped, count goes to 0, data is freed.
```


Closure (anonymous function)

```
fn main() {
```

```
    let x = 5;
```

```
    let result = |y| y * x;
```

```
    println!("Result: {}", result(3));
```

```
}
```


Rust concurrency

Threading based

Communication between threads can be via

- message passing (channel)
- Shared-state

Can customize via Sync and Send traits,

Threading

```
use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {} from the spawned thread!", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {} from the main thread!", i);
        thread::sleep(Duration::from_millis(1));
    }
}
```

Move ownership in Threading

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("Here's a vector: {:?}", v);
    }); //as read only, the compiler infers that
thread closure only borrows v, take no ownership

    drop(v); // oh no! As the thread function may
start later and access v now...

    handle.join().unwrap();
}
```

Default is borrow

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        println!("Here's a vector: {:?}", v);
    });

    handle.join().unwrap();
}
```

Solve the problem by explicitly move

Thread Communication by Channel

mpsc: multiple producers, single consumer

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || { //move the owner of tx to this thread
        let val = String::from("hi");
        tx.send(val).unwrap();
    });

    let received = rx.recv().unwrap();
    println!("Got: {}", received);
}
```

Safe Thread Communication by Channel

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("hi");
        tx.send(val).unwrap();
        println!("val is {}", val); //error
    });

    let received = rx.recv().unwrap();
    println!("Got: {}", received);
}
```

\$ error[E0382]: borrow of moved value: `val`

**The sending is really sent with val's ownership!
So, println access it causes an error coz the main
thread outside can modify it..**

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("hi");
        tx.send(val).unwrap();
        //NO ERROR!
    });

    let received = rx.recv().unwrap();
    println!("Got: {}", received);
}
```

Thread Communication by Shared-State

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(5);

    {
        let mut num = m.lock().unwrap();
        *num = 6;
    }

    println!("m = {:?}", m);
}
```

//m.lock() returns a smart pointer
MutexGuard (wrapped in something that you
call unwrap to get)

The smart pointer MutexGuard

- implements `Deref` to give you the content via `*`
- Implement `Drop` to auto release lock on leaving the scope

Error handling

No exception concept

Recoverable error

- E.g., File not found
- Mostly likely continue the program and report the problem to the user
- Returning `Result<T, E>` for you to recover and carry on

Unrecoverable error

- E.g., Access a location beyond the end of array
- = Bugs; Need to immediately stop the program
- `panic!` macro to stop execution when the program encounters an unrecoverable error.

Unrecoverable error and panic!

Sometimes, bad things happen in your code, and there's nothing you can do about it.

Either you call `panic!` yourself or the library, rust, OS, will call for you:

```
fn main() {  
    let v = vec![1, 2, 3];  
    v[99];  
}
```

Recoverable error

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

```
io::stdin().read_line(&mut guess).expect("Failed to read line");
```

Read_line returns a Result, which is an enum of either OK or Err (they are enum “variants”)

- Both OK and Error are object with an *.expect* method()

OK.expect() → return the value

Err.expect() → return the error message

But coding:

if OK

then return the value

else

call panic! is too verbose

unwrap() is a shortcut of that

unwrap()

```
let greeting_file_result = File::open("hello.txt");

let greeting_file = match greeting_file_result {
    Ok(file) => file,
    Err(error) => match error.kind() {
        ErrorKind::NotFound => match File::create("hello.txt") {
            Ok(fc) => fc,
            Err(e) => panic!("Problem creating the file: {:?}", e),
        },
        other_error => {
            panic!("Problem opening the file: {:?}", other_error);
        }
    },
};
```

vs

```
let greeting_file = File::open("hello.txt").unwrap();
```

rust

```
// A function that might fail
fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        // Err = failure, with an error message
        Err("cannot divide by zero".to_string())
    } else {
        // Ok = success, with the computed value
        Ok(a / b)
    }
}

fn main() {
    // Case 1: success
    let res1 = divide(10, 2);
    // Case 2: error
    let res2 = divide(10, 0);

    // Pattern-match on Result
    match res1 {
        Ok(value) => println!("10 / 2 = {}", value),
        Err(msg) => println!("Error: {}", msg),
    }

    // A more compact way with if let
    if let Err(msg) = res2 {
        println!("Second division failed: {}", msg);
    }
}
```


OO in Rust

Trait, Struct, Impl are all OO stuff in Rust

Impl can apply on a struct without a trait (interface) as their struct's own method

Rust has **no** formal class keyword though, use struct is enough

```
pub struct AveragedCollection {  
    list: Vec<i32>,  
    average: f64,  
}
```

```
impl AveragedCollection {  
    pub fn add(&mut self, value: i32) {  
        self.list.push(value);  
        self.update_average();  
    }  
  
    fn update_average(&mut self) {  
        let total: i32 = self.list.iter().sum();  
        self.average = total as f64 /  
        self.list.len() as f64;  
    }  
}
```

Function :: vs .Method

<https://stackoverflow.com/questions/51737045/what-is-the-difference-between-and-in-rust>

```
struct Rectangle {  
    width: u32, height: u32  
}  
  
impl Rectangle {  
    fn area(&self) -> u32 { self.width * self.height }  
}  
  
fn main() {  
    let rect1 = Rectangle { width: 30, height: 50, };  
    println!( "The area of the rectangle is {} square pixels.",  
        rect1.area() );  
}
```

.area() ⇒ method of an instance
(the method signature has &self)

```
impl Rectangle {  
    // Associated Function  
    fn square(size: u32) -> Rectangle {  
        Rectangle { width: size, height: size, }  
    }  
}
```

E.g, String::from, Rectangle::square
Like static function of that class
No call through an instance
That is called through namespace

Function :: vs .Method

```
struct Rectangle {  
    width: u32, height: u32  
}  
impl Rectangle {  
    fn area(&self) -> u32 { self.width * self.height }  
}  
  
fn main() {  
    let rect1 = Rectangle { width: 30, height: 50, };  
    println!( "The area of the rectangle is {} square pixels.",  
        rect1.area() );  
}
```

.area() ⇒ method of an instance
(the method signature has &self)

```
impl Rectangle {  
    // Associated Function  
    fn square(size: u32) -> Rectangle {  
        Rectangle { width: size, height: size, }  
    }  
}
```

E.g, String::from, Rectangle::square
Like static function of that class
No call through an instance
That is called through namespace

Refcell (interior mutability: dynamically check borrow rules at runtime ~~compile-time~~)

- If you break the rules, your program panics at runtime instead of failing to compile.

```
rust
use std::cell::RefCell;

fn main() {
    let x = RefCell::new(5);    // x is NOT mut

    {
        let mut x_ref = x.borrow_mut(); // runtime-checked mutable borrow
        *x_ref += 1;
    } // x_ref goes out of scope, mutable borrow ends

    let y = x.borrow();          // immutable borrow is fine now
    println!("x = {}", *y);      // prints 6
}
```

Key points:

- `x` itself is not `mut`, but we can still mutate the inner `i32` via `borrow_mut()`.
`rust-book.brown +1`
- `x.borrow_mut()` returns `RefMut<i32>`, which derefs to `i32`.
- When `x_ref` drops, the mutable borrow ends and you can borrow again. `mit`

If you try two mutable borrows at once, you get a panic:

```
rust
use std::cell::RefCell;

fn main() {
    let x = RefCell::new(5);
    let _a = x.borrow_mut();    // first mutable borrow is OK
    let _b = x.borrow_mut();    // panics at runtime
}
```

This is analogous to violating the usual “one `&mut` at a time” rule, but caught at runtime. `doc.rust-lang +1`

“!” means macro call

A function signature must declare the number and type of parameters the function has. Macros, on the other hand, can take a variable number of parameters: we can call `println!("hello")` with one argument or `println!("hello {}", name)` with two arguments. Also, macros are expanded before the compiler interprets the meaning of the code, so a macro can, for example, implement a trait on a given type.

Typical declarative macro:

```
macro_rules! square {  
    ($x:expr) => {  
        $x * $x  
    };  
}  
  
fn main() {  
    let result = square!(5);  
    println!("The square of 5 is:  
of 5 is: 25", result); // Output: The square  
}
```

rust

 Copy

```
#[derive(Debug)]
struct Person {
    name: String,
    age: u32,
}

// The generated code would look similar to this:
impl std::fmt::Debug for Person {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        f.debug_struct("Person")
            .field("name", &self.name)
            .field("age", &self.age)
            .finish()
    }
}
```

The Debug trait is part of the Rust standard library and is used to provide a human-readable string representation of a value, which is useful for debugging and logging purposes. The implementation generated by the `#[derive(Debug)]` macro will automatically create a `fmt` method that prints out the struct's field names and values in a formatted way.

So, when you have a `Person` instance and you print it using the `println!("{}", person)` syntax, the Rust compiler will use the automatically generated Debug trait implementation to produce the output, which might look something like this:

“!” means macro call

- Procedural macros are functions that take Rust code as input and generate new Rust code as output.
- They are defined using the `#[proc_macro]` attribute and are typically used for things like deriving traits, creating DSLs (Domain-Specific Languages), or transforming syntax.

Example:

Derive Macros:

These macros allow you to automatically generate trait implementations for your structs and enums. A common example is the `#[derive(Debug)]` macro, which generates the `Debug` trait implementation for a type.

```
#[derive(Debug)] //it automatically generates code that implement the Debug trait
struct Person {
    name: String,
    age: u32,
}
```

Derive - auto generate codes that implement a trait for you. I.e., you don't need to implement some common traits yourself

```
#[derive(PartialEq, Clone)]
struct Foo<T> {
    a: i32,
    b: T,
}
```

```
struct Foo<T> { a: i32, b: T }
impl<T: PartialEq> PartialEq for Foo<T> {
    fn eq(&self, other: &Foo<T>) -> bool {
        self.a == other.a && self.b == other.b
    }

    fn ne(&self, other: &Foo<T>) -> bool {
        self.a != other.a || self.b != other.b
    }
}
```

Derive - auto generate codes that implement a trait for you. I.e., you don't need to implement some common traits yourself

```
#[derive(Debug)]  
struct Foo<T> {  
    a: i32,  
    b: T,  
}
```

Then when println, say, a struct, can print in a formatted way for easy debug